

A GAME OF DECEPTION



October 23, 2025

“The one where we lure them in.”

Credits

Content: Muris Sladić, Sebastian Garcia, Veronica Valeros, Maria Rigaki, Martin Řepa, Lukáš Forst, Ondřej Lukáš

Illustrations: Fermin Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

Audio files, 3D prints, and Stickers: Veronica Valeros

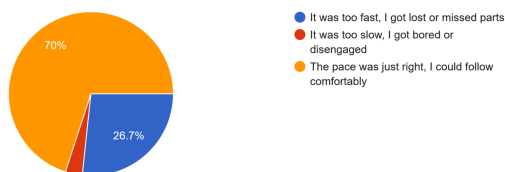
CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

Audio files, 3D prints, and Stickers: Veronica Valeros

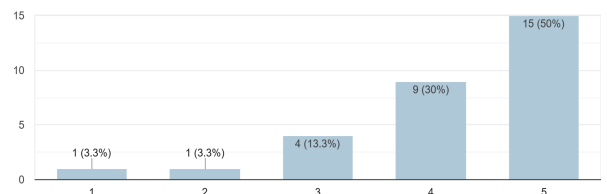
CLASS DOCUMENT	https://bit.ly/BSY2025-5
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroVideo
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from last class survey (14:32)

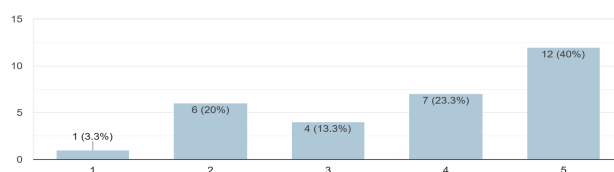
How was the pace of the class?
30 responses



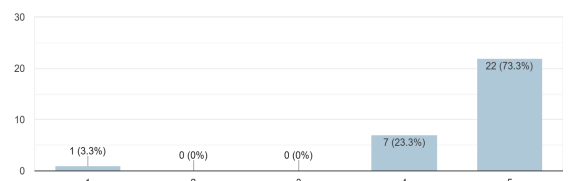
Were you able to follow what the teacher explained during class?
30 responses



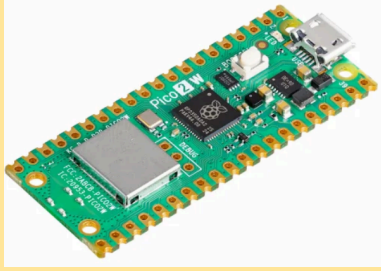
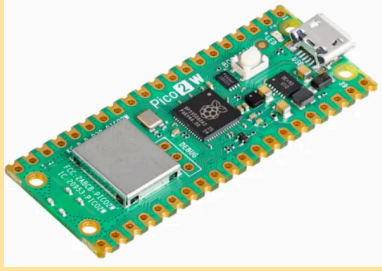
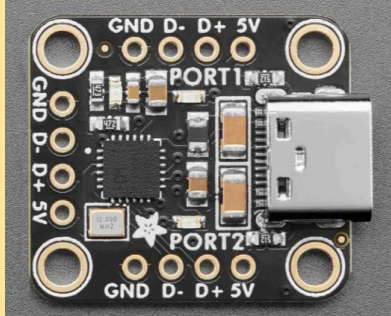
Were you able to follow along and run the commands during class on your device?
30 responses



How useful was the class document and other supporting resources shared by the teachers?
30 responses



Pioneer Prize for Assignment 4 (14:35)

1 st Place	2 nd Place	3 rd Place
 Raspberry Pi Pico 2 WH	 Raspberry Pi Pico 2 WH	 Adafruit Mini 2-Port USB Hub Breakout
strnaon2	kupkajan	seredda1

Parish Notices

- Remember that for CTU students, assignments 1, 2, 3, and 4 close on **October 30th, 23:59**. If you haven't solved them by the deadline, you won't be able to solve them.
- Don't forget to vote for the topic of the last class. We are very happy when we get your postcards 😊
- Deadline for Unlock Reports is December 11th
- FOR the NEXT CLASS, the TIME ZONE CHANGES IN PRAGUE from CEST (UTC+2) to CET (UTC+1). Please check your local time zone.
 - Class was 14.30 UTC+2 and now is 14.30 UTC+1
 - Maybe check <https://www.worldtimebuddy.com/>

The main topics in this class

1. Deception, why it works, key components
2. Cyber Deception & steganography for deception
3. Honeypots: honey systems, honey tokens, and honey services

Deception (14:40)

Goal: to learn what deception is, its basic concepts, applications, and why we use it in cyberspace.

What is deception?

Deception is an act of creating or maintaining in others a belief considered false.

There is a lot of ambiguity about what things are a deception. Is some behavior deception, or can it just be used for deception in some cases?

- There is no definitive answer.
- Opinions might and do differ

Deception is common in many areas of our society:

- 📰 Advertising, propaganda, and misinformation
 - False sales - saying discounts are up to 80%, but “up to” is in really small print
- ⚽ Sports
 - Faking the direction of movement in football, for example
- 🎭 Entertainment
 - Magicians - use of misdirection, repetition of the same thing, but in a different way
- 👤 Politics
 - Highlighting only the positive or negative side to shape public opinion
 - Presenting polls that fit your specific narrative; ex. Interviewing people on their opinion on some new policy and later showing just the responses that the policy makers like, in order to create the idea that is the opinion of the majority
- 🇺🇸 Military
 - Countless examples throughout human history

- Battle for the Wounded (Battle of Neretva) - https://en.wikipedia.org/wiki/Case_White
-  Internet
 - Social engineering
-  Animal world
 - Opossums playing dead
 - Fake poisonous frogs

Unlike lying, **deception does not require a statement to be made**, and **deception implies success**.

- Lying can be used for deception purposes, but it does not have to be.

Why does deception work?

- Some experiments show that the success rate for detecting lies and deception is barely above 50%¹
- Human and animal perception of the world and reality is based on patterns that are called **biases**².
- Biases influence decision-making, beliefs, and behavior.
 - **Cognitive biases:** Systematic patterns of deviation from norm and/or rationality in judgment.
 - Intrinsic to our nature in perceiving and processing information. It is impossible to avoid and occurs involuntarily.
 - Derivative of human processing limitations. Our brains are lazy, we employ shortcuts.
 - We tend to perceive what we **expect** to perceive.
- Are there other things besides biases that make us vulnerable to deception?
 - Emotions
 - Lack of Knowledge
 - Information Overload
 - Repetition - people are more likely to believe a false statement if they hear it repeatedly

¹ A. Vrij. Detecting lies and deceit: Pitfalls and opportunities. 2nd Edition. John Wiley & Sons, 2008

² <https://www.visualcapitalist.com/every-single-cognitive-bias/>

LESSON 5 / A GAME OF DECEPTION

- Poor critical thinking skills
- Social Pressure
- Discussion question. Do you think these things above are separate from biases?
 - Do they maybe cause our biases, or do they exist in addition to biases?
- The things mentioned above are all causes of Cognitive Bias
- Most often, it is probably a combination of multiple factors that makes us vulnerable.
- One of the most important things to remember about deception, so let's repeat
 - We tend to perceive what we **expect** to perceive.

Components, principles, and variants of deception

Deceptions may differ in their complexity, but some components are common to all:

- Deceiver & Target
- Deception story
- Channels
- Target feedback

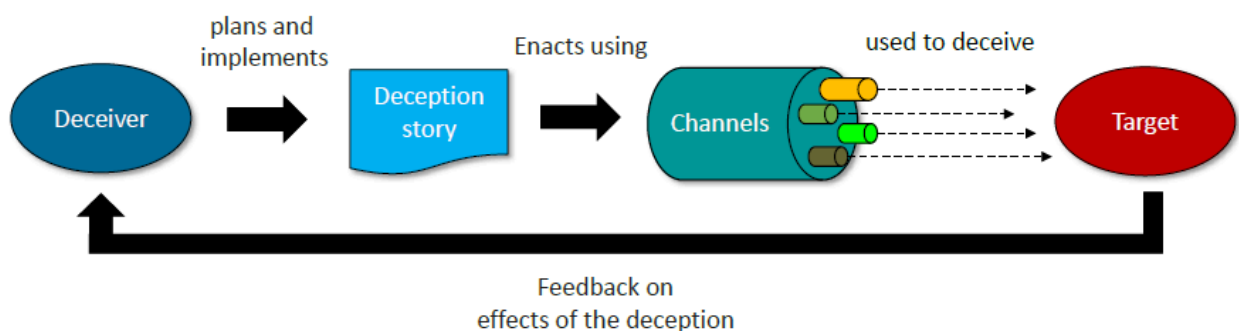


Figure 1. Strategic deceptive flow³

Have you heard of ANOM?

- FBI trojan horse to catch criminals
- https://en.wikipedia.org/wiki/Operation_Trojan_Shield
- Advertised to criminals as encrypted chat network
 - Shielded communication from law enforcement

³ C De Faveri, Modeling Deception for Cyber Security, 2021, https://run.unl.pt/bitstream/10362/148907/1/Faveri_2022.pdf

- Actually run by FBI
 - Operation Trojan Shield
 - From 2018 to 2021
- Why did criminals use it?
 - They used Phantom Secure before; Shut down in 2018
 - They needed a new app quickly

We can say that deception is governed by 4 basic **principles**:

- **Truth**: The default state of the world. Deception needs to be an exception to a non-deceptive world.
 - What was the truth in the ANOM example?
- **Denial**: Blocking the target's access to true data
 - What was denial in the ANOM example?
- **Deceit**: Lying and/or simulating
 - What was deceit in the ANOM example?
- **Misdirection**: Drawing attention away from real assets and activities
 - Was there misdirection in the ANOM example?
- **One extra: Confusion** - degrade perceptual abilities
 - Was there confusion in the ANOM example?

Variants of deception:

- **A-Type** (ambiguity-increasing) - target **unsure** of what to believe
 - *Example*: Deception target somehow has a list of access credentials to an important server. However, they found out or for some reason assume we implanted some fake credentials that would alert us of an attack if used. To the target all the credentials look equally plausible.
- **M-Type** (misleading) - target **believes in the false** alternative
 - *Example*: Deception target has a list of access credentials to an important server, which they believe to be completely authentic. However, the whole list is fake, and any of those credentials used would alert us of an attack.

Deception can be either:

- **passive** or **active**.

LESSON 5 / A GAME OF DECEPTION

- Active deception, unlike passive deception, requires us to interact with the victim and to adapt to the situation

Magruder's principles

Before attempting deception, it is vital to know:

- It is generally easier to induce a target to maintain a preexisting belief than to change it or create a new one.
- Following up on Magruder's principle, it has been asserted that
 - (A successful) Deception should contain 95% truth.

Recap: To create a good deception... Be imaginative and creative, and understand the need, the purpose, the setting, the target, and prepare well.

Cyber Deception (15:05)

Goal: to learn how deception is used in cyberspace.

What is cyber deception?

It is a form of deception that leverages digital tools and the online environment to deceive a target.

It can be used by both attackers and defenders.

How attackers can use deception

There are many examples of attackers using deception. Often they use techniques and tools that are not deceptive themselves in order to achieve deception. Some examples are:

- Social engineering
- Phishing
- Evil Twin - Websites
 - <https://tiffanycoshop.com> - DO NOT CLICK THIS LINK!
 - Copy the link in the shouldiclick page
 - <https://www.shouldiclick.org/> - protection
 - Fake DNS domains
- Evil Twin - Rogue WiFi access point (MitM)
- Deepfake technology - AI

- Steganography

😬 Steganography - There is definitely nothing suspicious going on

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- CTU students, let's go to our class Docker containers
- MOOC students, open the SCL
 - `apt update`
 - `apt install xxd -y`
- Let's download a photo
 - `wget "https://drive.google.com/uc?export=download&id=18j3-cIJzSCiZKTW5qzgfk06Yf0wm0qKX" -O Poirot-Books.jpg`
- Let's check the photo out by clicking the link.
 -  `Poirot-Books.jpg`
- There is more to this photo than it meets the eye
- `strings` is a Linux command to print the strings of printable characters in files.
 - `strings Poirot-Books.jpg`
- `xxd` creates a hex dump of a given file or standard input.
 - `xxd Poirot-Books.jpg`
- Now we see that photos can carry some hidden data.
 - In this case, it is a hidden text message
 - It can even be a file

Steganography is the practice of representing information within a medium in such a manner that the presence of the information is not evident to human or machine inspection.

- Today, in cyberspace, Steganography is detected with tools and ML, so it should also be resistant to that.
- One advantage of steganography is that fewer people try to find it and break it because they don't know something is there.
 - Good for situations where usual encryption may raise suspicion

LESSON 5 / A GAME OF DECEPTION

- However, it is security by obscurity and should never be the sole security solution.
- Another advantage is that it can use multiple types of media to transmit messages
 - Photos, video, text files, audio files, etc.
- Some disadvantages are
 - Limited capacity of data that can be hidden
 - Carrier quality degradation
- Here we reach that ambiguity again
- Steganography is not deception itself
- But it can be used for deception
 - Injecting a false belief that forbidden communication is not happening

How can attackers use steganography?

- Replacing pixels with malware code
 - Or just hiding malicious data
- Send instructions to command and control servers
- Hide stolen data
 - Unauthorized data transfers

What are the differences with encryption?

They are actually not even related but easily confused:

Steganography	Encryption
Steganography hides the fact that information exists	Encryption does not hide this.
Steganography can be encrypted (does not have to be).	
Steganography is designed not to be found.	Encryption is designed not to be broken

👁️ Let's continue with Steganography! When a photo is more than just a photo

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- We can analyze our photo further
 - ExifTool is free and open-source software for reading, writing, and manipulating image, audio, video, and PDF metadata.
 - `apt update`
 - `apt install libimage-exiftool-perl`
 - `exiftool Poirot-Books.jpg`
- Using exiftool, strings, and xxd we found some hidden messages in the photo.
- How to hide messages like in the example with *strings* and *xxd*?
 - `Echo New Message" >> Poirot-Books.jpg`
 - Remember, with Exiftool earlier, we found a message hidden in the comment section of jpg. You can write a message in a comment like this:
 - `exiftool -comment="New comment" Poirot-Books.jpg`
 - Running this command will rewrite the old comment
 - More ways can be found online, or ask ChatGPT ;)

Steghide⁴ - We can hide files in images too

Steghide is a tool to hide text messages in images. Very useful!

- `apt install steghide`
- `wget https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg`
- You can check the image by clicking on the URL
 - <https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg>
- Let's prepare file we want to hide
- `echo "hi there" > hide.txt`
- `steghide embed -ef hide.txt -cf Cat03.jpg`
 - `-ef` -file to hide

⁴ shetzl @chello.at, S.H. (2024). Steghide. [online] Sourceforge.net. Available at: <https://steghide.sourceforge.net/> [Accessed 22 Oct. 2024].

LESSON 5 / A GAME OF DECEPTION

- `-cf` -where to hide
- You will be prompted for a password
- Enter the password; It can be anything you want
- Remember the password
- Copy the stego-cat to your computer. **From your computer:**
 - CTU Students: Host Computer ▾
 - `scp -O -P <your_docker_port>`
`root@<YOUR-DOCKER-IP>:/root/Cat03.jpg Cat03.jpg`
 - `-O` - pass options to SSH; we use it here to pass the port number
 - `-P` - your SSH docker port
 - MOOC Students: Host Computer ▾
 - `scp -o StrictHostKeyChecking=no -o`
`UserKnownHostsFile=/dev/null -P 2222`
`root@127.0.0.1:/root/Cat03.jpg Cat03.jpg`
 - `-o` - pass options to SSH
 - `StrictHostKeyChecking=no` - do not confirm identity of the remote server
 - `UserKnownHostsFile=/dev/null` - do not store remote host's public key in known_hosts file
 - Remember the SCL password is: **ByteThem123**
 - See the image!
 - Does it look strange?
- Desteganzize (in your Dockers and SCL)
 - How to know if there is something hidden?
 - We either expect it or try to figure it out.
 - If we have the original photo, we can compare sizes.
 - `steghide extract -sf Cat03.jpg`

- `-xf output.txt` - if you want to save to a specific file

Briefly on Steganalysis

Steganalysis is the study of detecting messages hidden using steganography. It is analogous to cryptanalysis applied to cryptography.

Hard, but mostly it involves checking that the protocol/format was not respected or finding statistical anomalies regarding the use of parts of a protocol that should not be used so much (too many spaces in text, for example).

- Some things to look for
 - Inconsistencies in data compression
 - Distortion of neighboring pixels

It is so hard that companies sell it as a service⁵ ([Example](#))

Something for fun ✨

- During the break, put a hidden message for your class colleagues in a photo of your choice. ([Make sure the photo and the message do not violate any ethical guidelines of the class.](#))
- Put the photo in the Matrix “random” channel for your colleagues to analyze and read your message.

~~~~ 🧡 **First Break!** 🧡 ~~~~ (15:30)

## Deception for defenders

Why do defenders use deception in cybersecurity?

- Enhance protection & gather intelligence on attackers
- Deception can help us:
  - Get timely attack alerts
  - Track the attacker movement in our network

<sup>5</sup> WetStone Technologies (2020). StegoCommand™ [online] Available at: <https://www.wetstonetech.com/products/stegocommand-steganography-detection/> [Accessed 22 Oct. 2024].

## LESSON 5 / A GAME OF DECEPTION

- Learn attacker behavior
- Stall the attackers. Keep them away from real services
- Discover both outside and internal threats

There are many examples of deception used by defenders, for example:

- Honey Systems - decoy systems designed to attract and trap cyber attacks
  - Honeypots
  - Honeytokens
    - Honeyfiles
    - Honeywords
- Notifying about deception
  - How does this work?, you might ask
    - Do the attackers leave immediately?
  - Very interesting research by Ferguson et. al.<sup>6</sup>
- Some examples from real-life stories:
- Clifford Stoll
  - The Cuckoo's Egg
  - TED Talk: <https://www.youtube.com/watch?v=Gj8IA6xOpSk>

### Let's Explore Honey Tokens

- **Honey tokens:** mimic specific data or real data modified to inform about attack
  - Documents (PDFs, Word, etc), users, passwords (honeywords), tables in a DB, URLs, special credit card numbers etc.
- **Active Directory users<sup>7</sup>:** create and inject fake users in the Active Directory structure, luring attackers away from other targets. Early detection of attacks.

---

<sup>6</sup> Ferguson-Walter, K., Major, M., Johnson, C. and Muhleman, D. (2021). *Examining the Efficacy of Decoy-based and Psychological Cyber Deception* Examining the Efficacy of Decoy-based and Psychological Cyber Deception. [online] Available at: <https://www.usenix.org/system/files/sec21-ferguson-walter.pdf>

<sup>7</sup> Lukas, O. and Garcia, S. (2021). Deep Generative Models to Extend Active Directory Graphs with Honeypot Users. *Proceedings of the 2nd International Conference on Deep Learning Theory and Applications*, [online] pp.140–147. doi:<https://doi.org/10.5220/0010556601400147..>

- **Server users:** create fake users in a system and monitor logins to that account.
- **URL tokens**<sup>8</sup>: create fake URLs to use as early warning systems, detect insider threats, data exfiltration, and more:
- Let's visit this link and create a URL honey token
  - <https://canarytokens.org>
  - Token:  
<http://canarytokens.com/images/traffic/npqmbiiu59t2ar6d8qkgnojal/pos.t.jsp>
  - Logs:  
<https://canarytokens.org/nest/history/d26e9d336ea5d1e8d54992f79d07c1f1/npqmbiiu59t2ar6d8qkgnojal>
- Many, many more! Explore!

## Honeypots

Goal: to learn what honeypots are and how to install and use them.

Honeypots are "security resources whose value lies in being probed, attacked, or compromised"<sup>9</sup>. All honeypots share four key characteristics: they "are **deceptive**, **discoverable**, **interactive**, and **monitored**"<sup>10</sup>. They can be anything from a file to an entire operating system.

- **Any attempt** to communicate with the honeypot **is, by definition, an attack** since no legitimate users or applications should use the honeypot system<sup>11</sup>.
- "By watching attackers break into and control a honeypot, we learn how [they] operate and why"<sup>12</sup>

Honeypots allow defenders to:

- **Track attackers:** from commands to operators
- **Learn attackers' actions:** new attacks, new exploits, new strategies

<sup>8</sup> Thinkst Applied Research (2024). *Know. Before it matters*. [online] Canarytokens. Available at: <https://canarytokens.org/nest/generate> [Accessed 22 Oct. 2024].

<sup>9</sup> L. Spitzner (2002) Honeypots: Tracking Hackers. Addison-Wesley Longman Publishing Co., Inc.

<sup>10</sup> C. Sanders (2020). Intrusion Detection Honeypots: Detection Through Deception. Applied Network Defense.

<sup>11</sup> Supra note 2

<sup>12</sup> Supra note 1

- **Gain knowledge:** who attacks, where attacks come from, and what the attacks are

### Deploy your first honeypot!

Let's try to deploy our first honeypot from our containers and see what we can learn:

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Log in to your containers/SCL
2. Create a directory that we will share on the web server:
  - a. `mkdir /tmp/publicshare/`
  - b. `cd /tmp/publicshare/`
  - c. `echo "Not a Honeypot" > /tmp/publicshare/README.md`
3. Create a Python web server
4. CTU Students: Docker container. ▾
  - a. `tmux new-session -d -s FirstHoneypot bash -c 'python3 -m http.server 80 --directory /tmp/publicshare'`
    - i. `-d` → daemon (detached) mode
    - ii. `-m` → long formatting
    - iii. `-s` → tmux session name
    - iv. `bash -c ''` → command to execute on the screen
    - v. `python3 -m http.server 80` → simple HTTP server
    - vi. `--directory /tmp/publicshare` → share the content of this directory
5. MOOC Students: SCL HackerLab ▾
  - a. `tmux new-session -d -s FirstHoneypot bash -c 'python3 -m http.server 80 --directory /tmp/publicshare'`
6. CTU students, let's check each other's web servers. MOOC students, let's find web servers in your own Docker:



a. `nmap -p 80 -sV 172.20.0.0/24` - CTU Students: Docker container. ▾

i. `-p 80` → Only port 80

ii. `-sV` → Attempt to identify the service version

b. `nmap -p 80 -sV 127.0.0.1` - MOOC Students: SCL HackerLab ▾

7. Let's check our honeypot output:

a. `tmux a -t FirstHoneyPot`

From the logs of our web server, shown below, we can learn which IP attacked us, when the connections happened, which methods were used, what HTTP resources were requested, and what was the status code of the response.

```
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
172.16.1.68 - - [01/Nov/2023 14:50:31] "GET / HTTP/1.1" 200 -
172.16.1.68 - - [01/Nov/2023 14:52:18] "GET / HTTP/1.0" 200 -
172.16.1.68 - - [01/Nov/2023 14:52:18] code 501, message Unsupported method ('POST')
172.16.1.68 - - [01/Nov/2023 14:52:18] "POST /sdk HTTP/1.1" 501 -
172.16.1.68 - - [01/Nov/2023 14:52:18] code 404, message File not found
172.16.1.68 - - [01/Nov/2023 14:52:18] "GET /nmaplowercheck1698850338 HTTP/1.1" 404 -
172.16.1.68 - - [01/Nov/2023 14:52:18] "GET / HTTP/1.0" 200 -
172.16.1.68 - - [01/Nov/2023 14:52:18] code 404, message File not found
```

What are the problems and limitations of this basic honeypot?

- To start with, it runs as root, it is not separated from the real system, it is easy to identify as a honeypot, and logs are not stored. There are more problems.

## Characterizing Honeypots (16:15)

Honeypots, as we defined them, can take many forms. We can distinguish between:

- **Honey system:** mimics an entire operating system with all its underlying services
  - Linux or Windows OS, power grid control systems, navigation systems
- **Honey services:** mimics a specific protocol or software
  - SSH, SMB, RDP, Telnet, smart contracts, etc.
- **Honey tokens**

Honeypots can present various levels of interaction. We differentiate between:

- **Low interaction honeypots:** the honeypot allows attackers very limited actions. Lower value intelligence—low risk.
- **High interaction honeypots:** the honeypot allows attackers to perform any actions the same way as a real system would. High-value intelligence. High risk.
- **Medium interaction honeypots:** anywhere in between the other two extremes. This is often the most common type of honeypot, balancing intelligence collection and risk.

? What type and what interaction level did the honeypot we deployed before have?

## Let's Explore Honey Services

### Cowrie: Telnet and SSH Honeybot

"Cowrie is a medium to high interaction SSH and Telnet honeypot designed to log brute force attacks and the shell interaction performed by the attacker"<sup>13</sup>. It's highly customizable and probably one of the most used honeypots in the industry.

Cowrie can give us information about:

- **Who is attacking:** IP address, human, bot, etc.
- **What credentials** were used for attacking?
- **What actions** do the attackers attempt to do on the honeypot: commands, downloads, reverse shells, scripts, lateral movement, etc...

### Installing Cowrie

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

**CTU students, you will need Cowrie for Assignment 5, so you need to follow these configuration/installation steps.**

Let's install and run Cowrie in our containers/SCL to try and experience it:

- Install dependencies - already done in your Docker
  - `apt-get install git virtualenv libssl-dev libffi-dev build-essential libpython3-dev python3-minimal authbind`

<sup>13</sup> Cowrie (2015) Cowrie SSH/Telnet Honeybot [online] Available at: <https://github.com/cowrie/cowrie>. At the attack accessed on October 22, 2024.

- Add the non-root **cowrie** user
  - `adduser --disabled-password cowrie`
  - Press Enter for all fields and 'Y' at the end
- Change to user Cowrie: the rest of the actions will be run as this user **not as root**
  - `su - cowrie`
- Clone the Cowrie repository:
  - `git clone http://github.com/cowrie/cowrie`
  - `cd cowrie`
- Create a virtual Python environment:
  - `python3 -m venv cowrie-env`
  - `source cowrie-env/bin/activate`
- Install more packages
  - `python3 -m pip install --upgrade -r requirements.txt`
  - `pip install -e .`
- Create a Cowrie configuration file:
  - `cp etc/cowrie.cfg.dist etc/cowrie.cfg`
- Edit the new configuration file:
  - `vi etc/cowrie.cfg`
  - Enable Telnet:
    - Search for the [telnet] section (line 669)
      - In vi just press “:669” to go to line 669
    - Change `enabled = false` to `enabled = true`

```
652 # Enable Telnet support, disabled by default
653 enabled = true
654
```
- Start Cowrie

## LESSON 5 / A GAME OF DECEPTION

- `cowrie start`
- `/home/cowrie/cowrie/bin/cowrie start`
- There are other commands: `status`, `stop`, `restart`
- Cowrie will run in the background in ports 2222/TCP (SSH) and 2223/TCP (Telnet)
  - Check that it is running: `netstat -anp`
    - `-a` → show both listening and non-listening sockets
    - `-n` → numeric values
    - `-p` → show the associated program or service
  - Check that it is running: `ps afx`

### Playing with Cowrie

CTU students find a friend to attack you in the class. Share your IP address in the Matrix chat, and be sure somebody attacks you. If nobody attacks you, tell us! Get some IP to attack! MOOC students, attack yourself.

- Connect to Cowrie using SSH (user: root, password: *try any password*)
  - `ssh root@<friends' IP> -p 2222` CTU Students: Docker container. ▾
  - `ssh root@127.0.0.1 -p 2222` - MOOC Students: SCL HackerLab ▾
- Connect to a Cowrie using Telnet
  - `telnet <friends' IP> 2223` - CTU Students: Docker container. ▾
  - `telnet 127.0.0.1 2223` - MOOC Students: SCL HackerLab ▾
  - `ncat <friends' IP> 2223` - CTU Students: Docker container. ▾
  - `ncat 127.0.0.1 2223` - MOOC Students: SCL HackerLab ▾
    - use **root** as login
- Explore the system? What can you do? What did you find?

## Cowrie's Logs and TTY sessions

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

Cowrie's logs are located at `var/log/cowrie`:

- Access the logs:
  - `cd /home/cowrie/cowrie/var/log/cowrie`
    - `cat cowrie.log | less -S`
    - `cat cowrie.json |less -S`
- Search for new connections:
  - Search for “New connection” in less. Use /
    - `cat cowrie.log | less -S`
    - `/New connection`
  - `cat cowrie.log |grep -i "New connection" | less`
- See attacks in real-time:
  - `tail -f cowrie.log`

Cowrie also records the TTY sessions:

- Go to the Cowrie home folder:
  - `cd /home/cowrie/cowrie`
- Activate the virtual environment (only if you haven't done so already):
  - `virtualenv --python=python3 cowrie-env`
  - `source cowrie-env/bin/activate`
  - `pip install -e .`
- Check that you have sessions to replay:
  - `ls -lh var/lib/cowrie/tty/`
- Replay one of your sessions:

- `playlog var/lib/cowrie/tty/<id>`

~~~~ 💖 **Second Break!** 💖 ~~~~ (16:35)

What is the problem with honeypots?

- Identification and Fingerprinting^{14 15}
- Vulnerabilities
- Do they attract more attacks? Nobody knows
- Would you put it in your production servers? Probably not.



What is a good (cyber)deception? How do we measure it?

1. The one that works.
2. We do not know (exactly).

Ideally, we want attackers to waste their time in the honeypot and not work in the real system.

- **Question:** In this class, we worked with Cowrie. As an attacker, how would you recognize Cowrie?
- MOOC students, please write your answers to Matrix General chat. We will show them and discuss them.

How do attackers fingerprint honeypots?

- Attackers could be using tools to detect honeypots
 - Ex. <https://github.com/referefref/honeydet>
- Some honeypots have identifiable signatures, ex. Default Cowrie
- Traffic analysis
 - Specific ports - Cowrie 2222
 - Absence of legitimate traffic

¹⁴ Srinivasa, S., Pedersen, J.M. and Vasilomanolakis, E., 2021. Gotta catch'em all: a Multistage Framework for honeypot fingerprinting. *arXiv preprint arXiv:2109.10652*.

¹⁵ Vetterl, A. and Clayton, R., 2018. Bitter harvest: Systematically fingerprinting low-and medium-interaction honeypots at internet scale. 12th *USENIX Workshop on Offensive Technologies (WOOT 18)*. URL: <https://www.usenix.org/conference/woot18/presentation/vetterl>

- No services running on an open port
- Inside honeypot
 - A small number of files? (Could be)
 - Too juicy names of files? (Could be)
 - Commands do not return the expected output?
 - Is the machine a virtual machine?
 - There are scripts to check this
 - Meterpreter *checkvm* script
- Requires experience
- **Can we detect tools that detect honeypots?**
 - Thinkst Canary has a feature to detect Honeypot Scanners
 - <https://help.canary.tools/hc/en-gb/articles/18387238624157-Detecting-Honeypot-Scanners>

The hard part is: **We do not know what attackers are thinking when interacting with the system.** We can only assume.

- Based on their behavior, the commands they are taking
- If the attackers leave quickly, did they suspect the honeypot?
 - Maybe. Maybe there is another reason?
- How to Evaluate Honeypots:
 - Deploy honeypots on the internet and monitor logs
 - Mostly bot interactions
 - Do humans fall for deception? No way to know!
 - Design an experiment with humans and give them a survey later
 - We learn some reasons behind the decisions they take in the computer
 - Humans are **biased**
 - Just mentioning the word “honeypot” can induce false beliefs

Some qualities of deception:

- **Enticingness** - how attractive it is
- **Believability** - how genuine it looks; can it be distinguished from the real system
- **Variability** - how believability changes over time or when we reveal the deception mechanism
- **Monitorability** - we need to be able to monitor interactions with it
- **Non-interference** - **benign users must not touch deception by accident**

Many online attacks are automated, meaning they are performed by bots. Honeypots also need to monitor that.

Modifying Cowrie

Cowrie is highly configurable, and almost anything on it can be changed. Yes, including accepted users, passwords, commands, etc:

<https://github.com/cowrie/cowrie>

Let's change the passwords that are valid to enter:

- a. Go to the Cowrie home folder:

- `cd /home/cowrie/cowrie`

- b. First, copy the default User DB to a real one:

- `cp etc/userdb.example etc/userdb.txt`

- c. Second edit this file to add a new user:

- `vi etc/userdb.txt`

- Add a new line with:

- `bsy:x:*`

- d. Restart the Cowrie service to load the new file and its changes:

- `cowrie restart`

- e. Connect with SSH to **your** Cowrie:

- `ssh bsy@172.20.0.xx -p 2222` - CTU Students: Docker container. ▾

- `ssh bsy@127.0.0.1 -p 2222` - MOOC Students: SCL HackerLab ▾

? We can now log in as **bsy**. Is that enough?

- Does the user exist in the system?

Let's add a new user:

- When a new user is added, what files are changed? What do we see in the system?
 - `/etc/passwd`, `/etc/shadow`, `/home/user`
- Go to the Cowrie home folder:

- `cd /home/cowrie/cowrie`
- If we check GitHub readme, we see *honeyfs* has contents for fake file system
- Let's edit the `/etc/passwd` file
 - `nano honeyfs/etc/passwd`
 - Add the following line
 - `bsy:x:1001:1001:bsy:/home/bsy:/bin/bash`
- Let's edit `/etc/shadow` file
 - First, create a password hash
 - `openssl passwd -1 <password>`
 - `-1` - algorithm used for hashing. You can use also 6 for example
 - `nano honeyfs/etc/shadow`
 - Add the following line
 - `bsy:<HASH>:15800:0:99999:7:::`
 - `cowrie restart`
- Let's check the changes
- Connect with SSH to **your** Cowrie:
 - `ssh root@172.20.0.xx -p 2222` - CTU Students: Docker container. ▾
 - `ssh root@127.0.0.1 -p 2222` - MOOC Students: SCL HackerLab ▾
 - `cat /etc/passwd`
 - `cat /etc/shadow`
 - `ls /home`
- We need to add the user directory to home
 - This means we need to update the file system
 - `fsctl` is the command to modify the file system
 - The fake file system is simulated using [pickle.fs](#)

LESSON 5 / A GAME OF DECEPTION

- Cowrie simulates file system behavior without touching real OS files
- [pickle.fs](#) is at location `src/cowrie/data/fs.pickle`
- To add a new directory, we do:
 - `fsctl src/cowrie/data/fs.pickle`
 - `mkdir /home/bsy`
 - `CTRL + C`
- We need to restart Cowrie
 - `cowrie restart`
- Connect with SSH to **your** Cowrie again and check the changes.
- Did it work?

CTU students clean-up (after you finish assignment 5!!!)

- a. How to stop cowrie
 - `cowrie stop`
- b. How to exit a virtual environment
 - `deactivate`
- c. How to delete a virtual environment: just delete the directory
 - `rm -r cowrie-env`
- d. Delete the extra users!

Let's do an experiment

If you did the experiment and survey before the class you do not need to do it again

If you have not done this before the class, go to this website:

<https://shellsevaluation.thesis.stratosphereips.org/>

There we have a small experiment to assess how people interact with various shell environments.

There is a task for you to find a hidden crypto-wallet key without being detected.

There is a survey after you finish the task. Please do not close the window before you submit the survey.

? Is this a honeypot?

Let's take a look at something on the teacher's screen.

What can you tell us about this system?

- Is this a honeypot? If yes, what gave it away?

We can try some commands like:

- `ls`
- `cd Desktop`
- `touch testfile.txt`
- `echo "Let's try this!" >> testfile.txt`
- `cat testfile.txt`
- `cd /`
- `cat /etc/passwd`
- `cat ~/Desktop/test.txt`

🤖 Try some more. Play with it and share your thoughts or findings in the Matrix chat or in person if you are in the class.



AI View

shellLM is a honeypot framework to generate believable honeypot environments dynamically on the fly using Large Language Models (LLMs).¹⁶¹⁷

- The user never interacts with a real system.
- LLM can be taught to mimic any service.

¹⁶ Sladić, M., Valeros, V., Catania, C., & Garcia, S. (2024, July). Llm in the shell: Generative honeypots. In 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW) (pp. 430-435). IEEE.

¹⁷ Sladić, Muris (2023) LLM in the Shell: Generative Honeypots Demo [online] Available at: <https://www.youtube.com/watch?v=0ysdHanr-jA>. Accessed on November 1, 2023.

- The more real the behavior mimicked by the LLM, the more users interact with it.

We can go beyond shellLM. How about a tool for Honey services? Well, lucky for you, we have it. It is called **VellMes** (Slavic god of, amongst other things, trickery -Veles; + LLMs). shellLM is a part of it, but also MySQL, POP3, and HTTP.

- <https://github.com/stratosphereips/VellMes-AI-Honeypot/tree/main>

Recap: What makes a successful honeypot?

A honeypot is successful when attackers probe it, attack it, and access it. Honeypot deception can be planned using the see-think-do deception methodology¹⁸.

- **SEE:** we want attackers to find and see the honeypot services, systems, and tokens.
- **THINK:** We want attackers to think these honeypots are valuable targets.
- **DO:** We want attackers to use the systems to do some interactions with the systems.

If defenders control all three of these, they have a high chance of detecting intruders early and increasing the intelligence collected from attackers.

Extra: Honeypots we love

- All-in-one [T-Pot](#): multi-honeypot containerized framework. Dozens of honeypots. IDS/IPS. Monitoring¹⁹.
- [Dionaea](#): is a low-interaction honeypot that captures attack payloads and malware²⁰
- [Glutton](#): proxy honeypot to log all attacks between the internet and other honeypots²¹
- All-in-one [Chameleon](#): customizable honeypots to gather intelligence. 19 honeypot services are supported.

¹⁸ JP 3-13.4 (2012) Military Deception [online] <https://info.publicintelligence.net/ICS-MILDEC.pdf>. Accessed on 11/01/2023.

¹⁹ Installing T-Pot Honeypot Framework in the Cloud – Stratosphere IPS, <https://www.stratosphereips.org/blog/2020/10/10/installing-t-pot-honeypot-framework-in-the-cloud>

²⁰ Dionaea – Catching bugs – The Honeynet Project, <https://www.honeynet.org/projects/active/dionaea/>

²¹ An analysis of Glutton – All Eating honeypot | by Muhammad Tayyab Sheikh (CS Tayyab) | Medium, <https://cstavyab.medium.com/an-analysis-of-glutton-all-eating-honeypot-625adf70a33b>

FOR the NEXT CLASS, the TIME ZONE CHANGES IN PRAGUE from CEST (UTC+2) to CET (UTC+1). Please check your local time zone.

Class was 14.30 UTC+2 and now is 14.30 UTC+1

Maybe check <https://www.worldtimebuddy.com/>

(17:35)

Assignment 5 (6 Points)

1. You must have a Cowrie installation in your Docker container running **SSH on port 2222** (Like we did in class).
2. Attackers will try to connect to this port and detect if it is a honeypot or a real service.
3. You need to try and make the honeypot look more realistic.
4. More details in [CTFd](#)



Class Feedback

By providing us with feedback after each class, you can help us make the next class even better!

<https://bit.ly/BSY-Feedback>



Side Dishes

More on Steganography: History and uses

- The first reference was in 440 BC in Greece when Herodotus mentions two examples in his [Histories](#). Histiaeus sent a message to his vassal, Aristagoras, by shaving the head of his most trusted servant, "marking" the message onto his scalp, and then sending him on his way once his hair had regrown.
- Mostly military use since then.
- Invisible ink.
- Morse code in yarn knitting. Common in WWII. [Here](#).
- In documents and prints, the message could be hidden by using two or more different typefaces, such as normal or italic.
- During and after World War II, espionage agents used photographically-produced [microdots](#) to send information back and forth.

- Velvaley Dickinson was a US citizen spying for Japan. She embedded info in the manufacturing orders of dolls shipped to Argentina. It was discovered when the Argentinian moved, and the letters bounced back.²²
- [Jeremiah Denton](#) repeatedly blinked his eyes in Morse code during the 1966 televised press conference that he was forced into as an American prisoner-of-war by his North Vietnamese captors, spelling out "T-O-R-T-U-R-E".
- 911 Attack: "bin Laden and others 'are hiding maps and photographs of terrorist targets and posting instructions for terrorist activities on sports chat rooms, pornographic bulletin boards and other websites, U.S. and foreign officials say.'"
 - <https://www.wired.com/2001/02/bin-laden-steganography-master/>
- And in your home, too
 - Some modern **printers** use steganography, including Hewlett-Packard and Xerox brand color laser printers. The printers add tiny yellow dots to each page. The barely visible dots contain encoded printer serial numbers and date and time stamps.
(<https://www.eff.org/press/archives/2005/10/16> , and <https://w2.eff.org/Privacy/printers/docucolor/>)



- Malware Attacks in 2021!
 - Malware is hidden inside images. ([Example](#))
 - Used steganography to embed RATs within the embedded images.
- LokiBot: The Famous Image Steganography Attack
 - LokiBot malware uses steganography to hide its malicious files.

²² https://en.wikipedia.org/wiki/Velvaley_Dickinson

LESSON 5 / A GAME OF DECEPTION

- The malware installs itself as two files: a .jpg file and a .exe file.
 - The .jpg file opens, unlocking data that LokiBot needs when implemented.
 - The malware places the image and the .exe file into a directory that it creates, along with a Visual Basic script file that runs the LokiBot file.
 - The script uses a decryption algorithm to extract the encrypted code from the image, enabling the VBScript file interpreter to execute the malware.
- Malware in CSS
 - It fetches a remote file called fonts.css, from which it retrieves JavaScript code ([Example](#)).
 - But how? The .css file contained a few lines of valid CSS code, but there were also non-visible characters such as spaces, tabs, and newlines.

```
/* latin */
@font-face {
  font-family: 'Open Sans';
  font-style: normal;
  font-weight: 700;
  src: local('Open Sans Bold'), local('OpenSans-Bold'), url
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-0
```


Snow

It is a tool to hide **text** *in text*!

- `apt install stegsnow`
- `echo "oh my god" > in.txt`
- `stegsnow -C -m "Good Morning" -p "pass" in.txt out.txt`
 - `-m` message
 - `-p` password
- Can you see where the info is stored?
 - `cat out.txt`
- What about
 - `xxd -C out.txt`
- Lets read it
 - `stegsnow -C -p "pass" out.txt`

Differences with polyglots

What are polyglots? A person who can speak many languages. Well, in our case, it is a *file* that includes many languages and can be read/*processed* by many different tools.

For example, a **PDF** document that is also a **ZIP** archive and a **Bash script** that runs a **Python webserver** that hosts **Kaitai Struct's WebIDE**, which allows you to **view the file's own annotated bytes**. ([Here](#))

Or a **PDF** that's also a valid **ZIP** and a valid **firmware** for the Apollo Guidance Computer. ([Here](#)).

Let's try a hands-on example with an image:

- We will analyze the following image:
<https://twitter.com/David3141593/status/1057042085029822464>
- Download it to your container:

LESSON 5 / A GAME OF DECEPTION

- `wget`
`"https://pbs.twimg.com/media/DqteCf6WsAAhqwV?format=jpg&name=120x120" -O image.jpg`
- Check the type of file downloaded:
 - `file image.jpg`
- Find the content by analyzing it byte by byte with binwalk
 - `binwalk image.jpg`
- Unzip the JPG
 - `mkdir test`
 - `mv image.jpg test`
 - `cd test`
 - `zip -FFv image.jpg --out image-fix.jpg (to fix)`
 - `unzip image-fix.jpg`
- Unrar the files that you got from the ZIP that you got from the JPG
 - `unrar x shakespeare.part001.rar`

Heralding: Multiservice Credential Harvesting

Heralding²³ is a low-interaction honeypot dedicated to credential harvesting across 13 different services, including SSH, Telnet, HTTP, POP3, etc.. Let's try it in our dockers!

- Install dependencies (already done in your containers)
 - `apt install libpq-dev`
- Clone the repository

²³ Johnnykv (2016) heralding: Credentials catching honeypot [online] Available at: <https://github.com/johnnykv/heralding>. Accessed on October 24, 2024.

- `git clone https://github.com/johnnykv/heralding.git`
- `cd heralding`
- Create a virtual Python environment²⁴:
 - `python3 -m venv heralding-env`
 - `source heralding-env/bin/activate`
- Install more packages
 - `pip install -r requirements.txt`
- Install heralding
 - `pip install .`
- Copy and edit the configuration file
 - `cp heralding/heralding.yml .`
 - `vim heralding.yml`
 - In the SSH section change the port to 2228
- Make the binary executable:
 - `chmod +x bin/heralding`
- Start the honeypot
 - `screen -d -m -S heralding bash -c '/root/heralding/bin/heralding'`
- Let's attack each other:

²⁴ Python documentation. (2024). *venv – Creation of virtual environments*. [online] Available at: <https://docs.python.org/3/library/venv.html> [Accessed 24 Oct. 2024]

LESSON 5 / A GAME OF DECEPTION

- `ssh user@172.20.0.xx -p 2228`
- `ftp 172.20.0.xx`
- `ncat 172.20.0.xx 25`
- Try other services: telnet, http, https, pop3, pop3s, imap, imaps, vnc, postgresql, and socks5.
- View the logs:
 - `tail -f log_auth.csv`
 - `tail -f log_session.json`